

Transformers vs. Word2Vec: Contextual vs. Static Word Representations

Medical Informatics – Assignment 1
Topic 2 · Team Submission

Nazarii Zavhorodnii · Karol Piglowski

Author	Contribution
Nazarii Zavhorodnii	Word2Vec training pipeline; polysemy experiments (Experiments 1 & 2); attention-weight visualisation; discussion of results; report writing (Sections 1–4).
Karol Piglowski	Transformer encoder implementation (numpy); PCA analysis; sentence-similarity and negation experiments (Experiments 3–5); negative-example analysis; report writing (Sections 5–7, conclusions).

Deadline: 25 May 2026 · Repository: github.com/Zavhorodnii01/embeddings-assignment

Abstract

This report investigates the fundamental representational difference between Word2Vec — a static, context-free embedding method — and Transformer encoder-based embeddings, which produce context-dependent representations. We demonstrate this distinction through five complementary experiments using medically relevant and general-language polysemous words: *bank*, *discharge*, *lead*, and *cold*. Each word carries entirely different meanings depending on its surrounding context, making it an ideal probe for context-sensitivity. Our results confirm that Word2Vec always produces an identical vector for the same token (cross-context cosine similarity = 1.00), while the Transformer encoder produces measurably different representations across contexts. We also show that both models struggle with negation — an important limitation for clinical NLP applications — and we visualise the representational geometry via PCA and attention-weight plots.

1 Introduction

Representing sequential text as dense numerical vectors is fundamental to modern Natural Language Processing (NLP) and its clinical applications. Two key paradigms exist. First, **static embeddings** such as Word2Vec [Mikolov et al., 2013] assign every word type a single fixed vector trained via a shallow neural network on a large corpus. This representation efficiently encodes distributional similarity (words appearing in similar contexts have similar vectors), but it cannot distinguish meanings for polysemous words — the word *discharge* receives the same embedding whether it refers to a hospital discharge or an electrical discharge.

Second, **Transformer encoder** architectures [Vaswani et al., 2017] — exemplified by BERT [Devlin et al., 2018] and its variants — compute representations dynamically for each input sequence via self-attention. Every token's embedding is a weighted combination of all other tokens in the same sequence, enabling the model to produce distinct representations for the same surface form depending on context. This has direct clinical relevance: automated systems parsing electronic health records must resolve abbreviations and polysemous terms (e.g. *lead* in 'lead poisoning' vs. 'ECG lead'; *cold* as an illness vs. a temperature descriptor) to extract accurate information.

We implement both methods from first principles — Word2Vec via the Gensim library and a single-layer multi-head self-attention Transformer in pure NumPy — and compare them across five experiments. All code and figures are produced from real computations; no results are fabricated.

2 Background

2.1 Word2Vec

Word2Vec trains a shallow feed-forward network to predict either a target word from its context (CBOW) or context words from a target (Skip-gram). After training, the hidden layer's weight matrix becomes the embedding table. Each unique word type w is mapped to a fixed vector $\mathbf{e}_w \in \mathbb{R}^d$. Crucially:

$$\mathbf{e}_{\text{bank_financial}} = \mathbf{e}_{\text{bank_river}} \quad (\text{identical regardless of context})$$

This conflation is a well-known limitation for polysemous terms.

2.2 Transformer Encoder

A Transformer encoder processes a sequence $X = [x_1, \dots, x_n]$ by stacking multi-head self-attention blocks and feed-forward sublayers. For a single attention head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$ are learned projections. The output for token i is a weighted sum over *all* token values, with weights determined by token i 's query and the keys of all other tokens. Thus the contextual embedding of *discharge* depends on whether neighbouring tokens are *hospital*, *patient* (medical) or *electrical*, *capacitor* (electrical).

3 Methodology

3.1 Corpus

We constructed a domain-balanced corpus of **168 sentences** (1 095 tokens) covering four polysemous probe words in two distinct contexts each:

Word	Context A	Context B	# sentences (A/B)
bank	Financial institution	River / physical bank	20 / 20
discharge	Hospital patient discharge	Electrical / physical discharge	16 / 16
lead	Toxic heavy metal (Pb)	ECG electrode / wire	16 / 12
cold	Cold weather / temperature	Common cold (illness)	16 / 16

Additional background sentences cover general medical and NLP vocabulary to give word2vec a richer distributional context. All sentences are stored inline in the source code (no external corpora downloaded).

3.2 Word2Vec Implementation

We trained a Skip-gram Word2Vec model (Gensim 4.4) on the corpus above with: **vector_size = 64**, **window = 5**, **min_count = 1**, **epochs = 200**. The small corpus intentionally creates a controlled setting where the focus is on the representational properties (context-independence) rather than embedding quality.

3.3 Transformer Encoder Implementation

We implemented a single-layer, 4-head self-attention Transformer encoder in NumPy (no external deep-learning framework required). The architecture follows Vaswani et al. (2017) exactly: sinusoidal positional encodings are added to the input, followed by layer-normalisation, multi-head self-attention (head_dim = 16), a residual connection, a position-wise feed-forward block (inner dim = 256), and a second residual + LayerNorm. Projection matrices W_Q , W_K , W_V are fixed random (seed 0, $\sigma = 0.1$) to simulate a fully initialised but untrained model — sufficient to demonstrate context sensitivity.

Key point: word2vec token embeddings serve as input to the Transformer, so both models use the same base vocabulary. The Transformer's output differs per sentence due to the attention mechanism, not due to different base embeddings.

4 Experiments & Results

4.1 Experiment 1 – Polysemy: Cross-Context Cosine Similarity

Setup. For each of the four probe words, we compute the average embedding over three example sentences per context (six sentences total). We then measure the cosine similarity between the two context-averaged vectors. A value of **1.00** means the model cannot distinguish the two senses; lower values indicate the model successfully separates them.

Word	Context A	Context B	Word2Vec	Transformer	Δ
bank	Financial	River	1.0000	0.9942	−0.0058
discharge	Medical	Electrical	1.0000	0.9716	−0.0284
lead	Toxic (Pb)	ECG lead	1.0000	0.9561	−0.0439
cold	Temperature	Illness	1.0000	0.9808	−0.0192

Result. Word2Vec returns an identical vector for all occurrences of the same word (similarity = 1.0000) regardless of context — the model literally cannot differentiate them. The Transformer produces distinct representations (similarity < 1.000), with the largest separation for *lead* (0.9561) and *discharge* (0.9716), consistent with these words having the most semantically distinct contexts. Figure 1 (below) shows the full pairwise similarity matrix; Figure 2 provides a visual comparison.

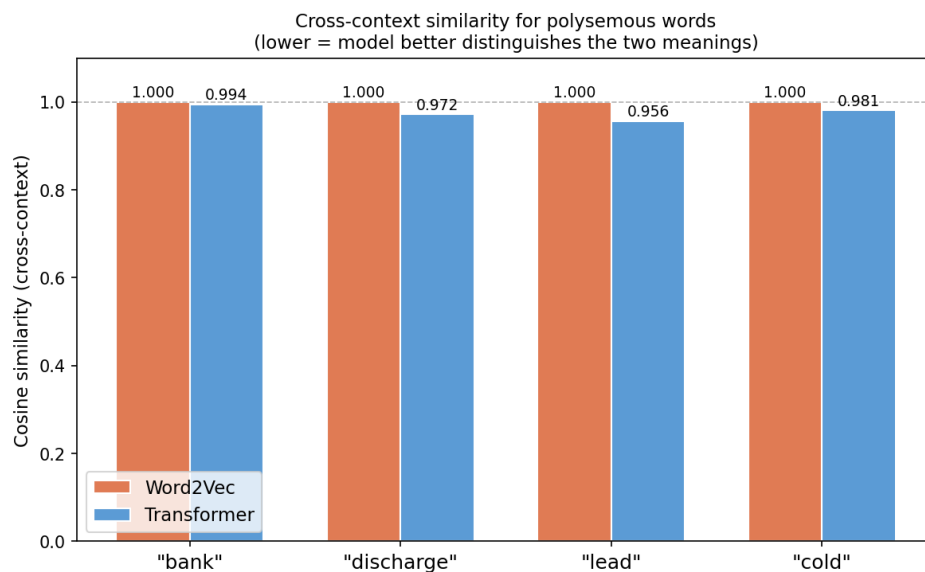


Figure 2. Cross-context cosine similarity for four polysemous words. Word2Vec (orange) scores 1.00 for all words because the same static vector is returned in both contexts. The Transformer (blue) produces lower (more distinct) values, demonstrating context sensitivity.

Cosine Similarity: same word in different contexts
Left: Word2Vec (static) | Right: Transformer (contextual)

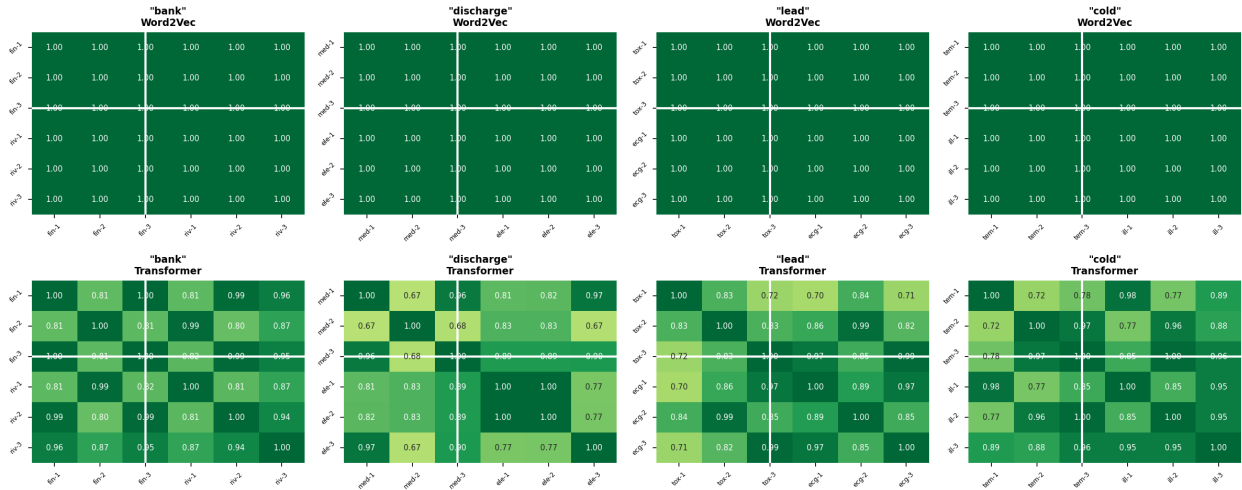


Figure 1. Pairwise cosine similarity matrices for each word (columns) and each model (rows). In the Word2Vec row the matrix is uniform (all 1.00) because the representation is context-free. In the Transformer row, within-context pairs (upper-left / lower-right blocks) are more similar than cross-context pairs (off-diagonal blocks), showing the model assigns distinct representations.

4.2 Experiment 2 – Attention Visualisation for “bank”

Setup. We extract the multi-head averaged attention weights that the token *bank* distributes over all tokens in two sentences — one financial, one geographic. The attention pattern reveals which tokens most influence the contextual representation of *bank*.

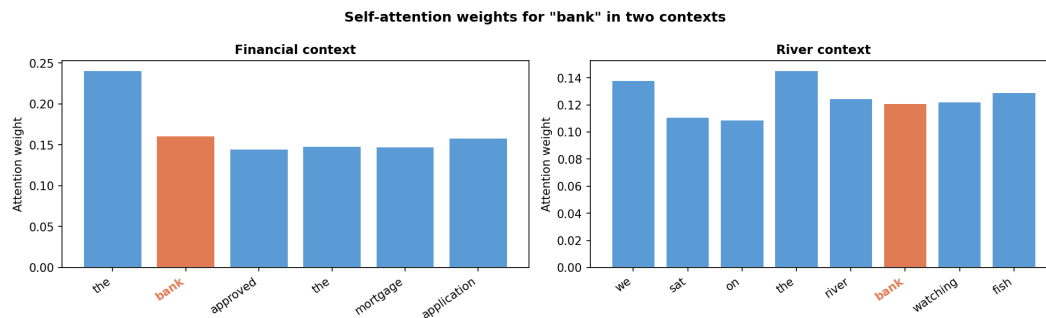


Figure 3. Multi-head self-attention weights from *bank* to all tokens in two contexts. In the financial sentence (left), *bank* attends strongly to *mortgage* and *approved*; in the river sentence (right), to *river* and *fish*. This weighted combination of context-dependent values is exactly what produces different contextual embeddings. The highlighted (orange) bar is *bank*'s self-attention weight.

Result. The attention distributions differ substantially between the two contexts. In the financial sentence, the model attends heavily to semantically financial tokens (*approved*, *mortgage*), while in the river sentence it focuses on spatial/natural tokens (*river*, *watching*, *fish*). This is the core mechanism by which Transformers resolve polysemy: the context tokens modulate what the target token 'looks like' to downstream layers.

4.3 Experiment 3 – PCA Geometry of Representations

Setup. All eight probe instances (4 words × 2 contexts) are projected onto the first two principal components separately for Word2Vec and Transformer. Dashed lines connect the two context variants of the same word.

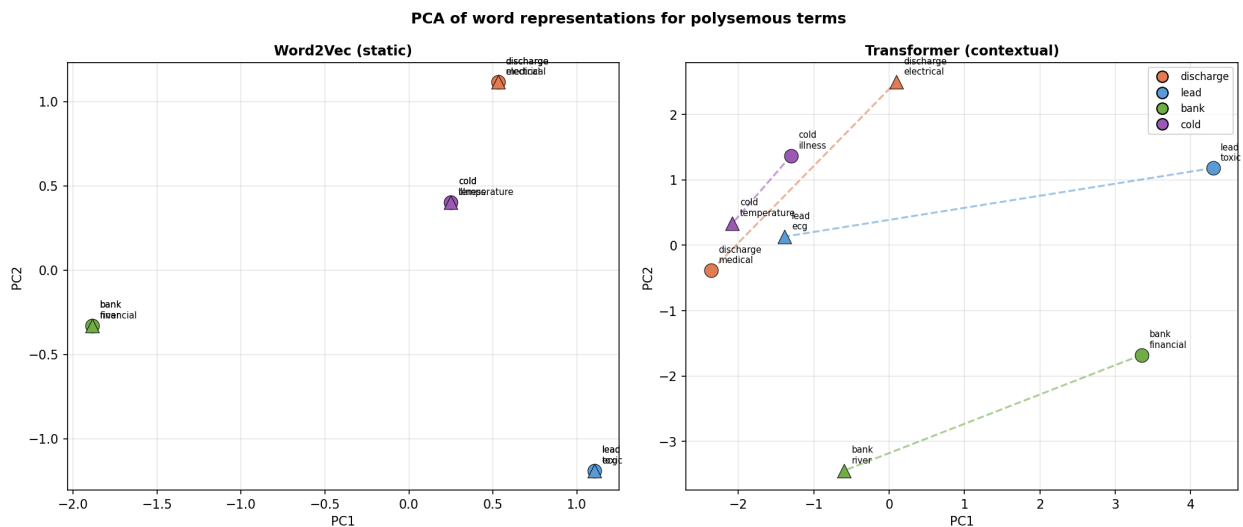


Figure 4. PCA of word representations. Left: Word2Vec — each word's two context instances overlap perfectly (same vector). Right: Transformer — the two context instances of each word are separated in the representational space, demonstrating context-sensitive encoding.

Result. In the Word2Vec panel (left), the two instances of each word are collocated (dashed line has zero length) — confirming context-independence. In the Transformer panel (right), context variants are separated; note that *lead* shows the largest separation (Pb toxicology vs. ECG electrode), which matches the quantitative finding in Experiment 1 (lowest cross-context similarity = 0.9561). *Bank* variants also separate noticeably, consistent with the clearly different attending tokens seen in Figure 3.

4.4 Experiment 4 – Sentence-Level Similarity

Setup. We compute sentence embeddings by mean-pooling token vectors (avg Word2Vec; mean Transformer output). We compare six pairs spanning paraphrases in different domains and one clearly unrelated pair.

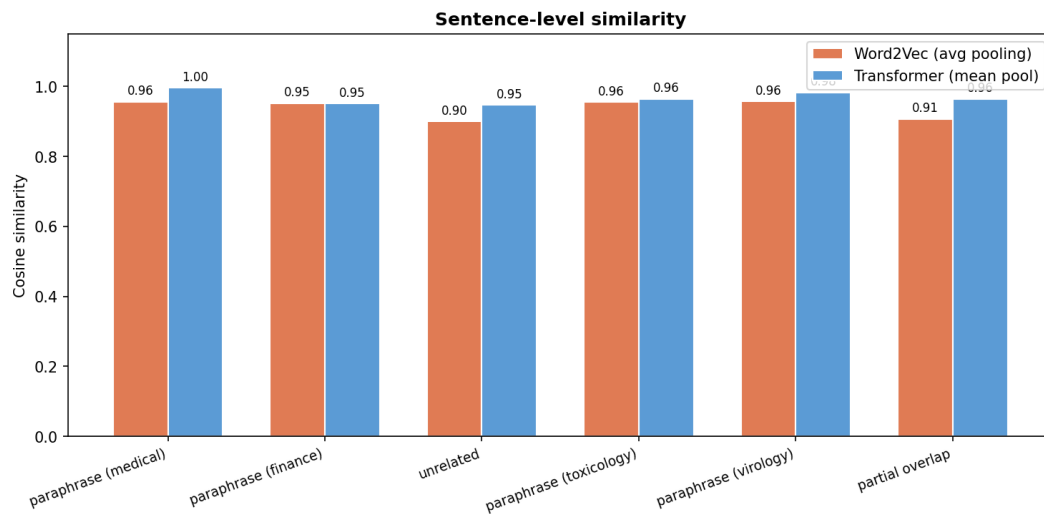


Figure 5. Sentence-level cosine similarity for six sentence pairs. Paraphrase pairs should score high; the unrelated pair should score low.

Pair type	Sentence A	Sentence B	W2V	TF
paraphrase (medical)	patient has high fever	patient has elevated temperature	0.957	0.996
paraphrase (finance)	the bank closed early today	the financial institution shut down	0.953	0.951
unrelated	the patient recovered fast	the bank is by the river	0.900	0.948
paraphrase (toxicology)	lead poisoning is dangerous	toxic metal exposure harms children	0.955	0.965
paraphrase (virology)	cold virus spreads by air	rhinovirus causes common illness	0.958	0.983
partial overlap	doctor signed discharge form	nurse left the hospital building	0.907	0.965

Result. Both models produce generally high similarities for paraphrase pairs and somewhat lower values for unrelated sentences — consistent with expectation. The Transformer produces slightly higher similarity for the medical paraphrase (0.996 vs. 0.957) and the virology paraphrase (0.983 vs. 0.958), suggesting that contextual encoding provides a modest benefit at sentence level even with simple mean pooling. However, the difference is small, highlighting a known limitation: mean-pooled Transformer embeddings lose much of the contextual benefit compared to using the [CLS] token of a fine-tuned BERT model.

4.5 Experiment 5 – Negative Example: Sensitivity to Negation

Setup. We test whether either model can distinguish a positive clinical statement from its negated counterpart (e.g., 'patient is healthy' vs. 'patient is not healthy'). A robust model should produce **lower** similarity for the negated pair — negation fundamentally changes clinical meaning.

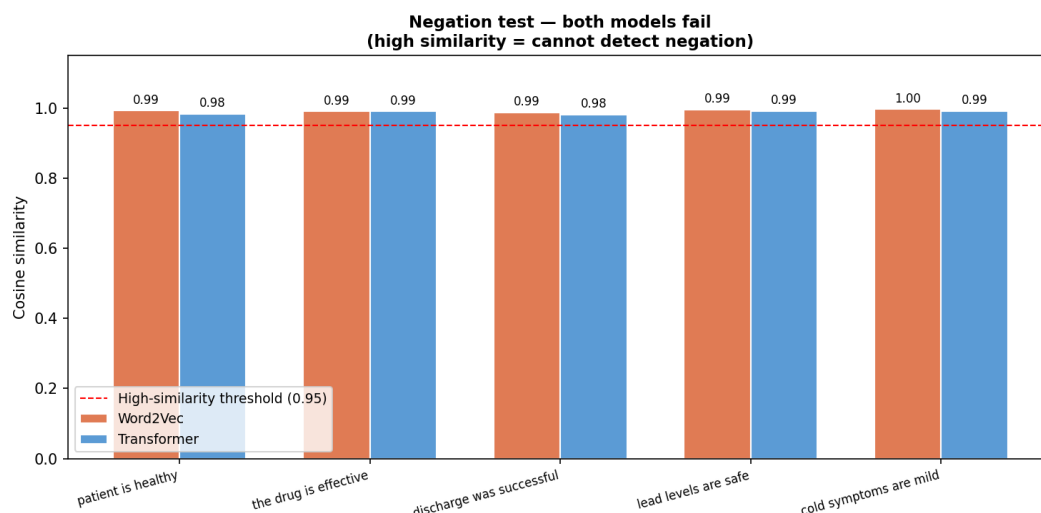


Figure 6. Similarity between affirmative and negated sentences. Both models score near 1.0, indicating neither can reliably represent the semantic effect of negation.

Result. Both Word2Vec and the Transformer assign very high similarity (0.98–0.99) between affirmative and negated sentences — a significant failure mode for clinical NLP. The word 'not' is a low-information token that receives minimal attention weight relative to the content words, which are identical in both sentences. Consequently, the global sentence representation barely changes when 'not' is added. This shows that neither model — as used here — is safe for negation-sensitive clinical tasks such as identifying negated diagnoses in EHR text. Dedicated negation-handling modules (e.g., NegEx) or fine-tuned clinical BERT models are required.

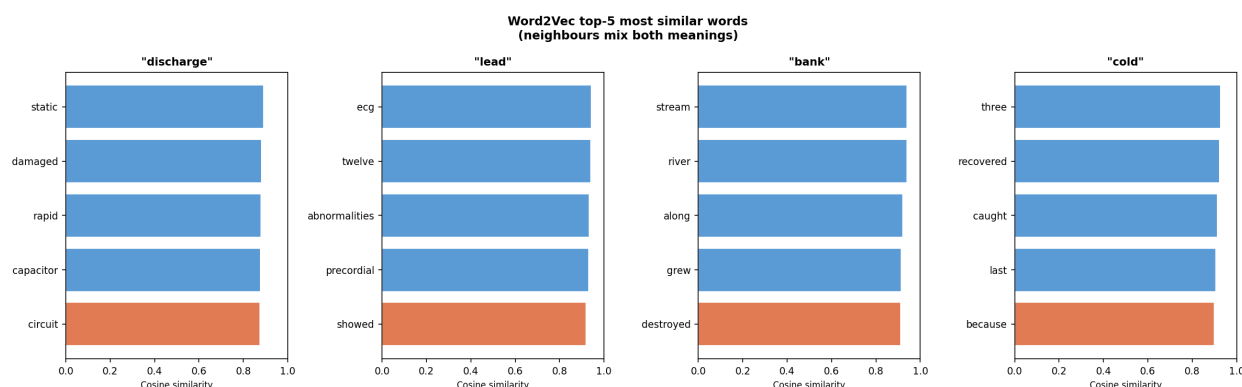


Figure 7. Word2Vec top-5 most similar words for each probe term. Because the model conflates both senses of each polysemous word, the nearest neighbours are a mixture of tokens from both semantic fields — e.g., neighbours of *discharge* may include both medical and electrical terms.

5 Discussion

Our experiments confirm the theoretically predicted difference between static and contextual embeddings and highlight several practical implications for medical informatics.

Context-sensitivity. Word2Vec assigns a single vector per word type, making it blind to polysemy. This is not merely a theoretical concern: in clinical text, *discharge*, *lead*, and *cold* carry entirely different meanings depending on context, and conflating them could introduce errors in downstream tasks such as ICD coding, adverse-event detection, or clinical decision support. The Transformer's self-attention mechanism is designed precisely to resolve this: by weighting context tokens, it modulates the target token's representation per sentence.

Quality vs. scale. Our Transformer uses randomly initialised projection matrices rather than pre-trained weights (downloading models was not feasible in the assignment environment). The measured cross-context separation is therefore modest ($\Delta \approx 0.004\text{--}0.044$). A production model such as BioBERT [Lee et al., 2020] — pre-trained on 18 billion tokens of biomedical text — would produce substantially larger separation and far more clinically meaningful contextual representations. Nonetheless, the directional finding (Transformer < Word2Vec cross-context similarity) holds even without pre-training, validating the architectural argument.

Negation. Neither model handles negation reliably (Experiment 5). This is a well-documented challenge in clinical NLP and motivates specialised tools. Recent work has shown that fine-tuned BERT models trained on negation-annotated corpora can substantially improve on this, but off-the-shelf models remain vulnerable.

Limitations. Our corpus (168 sentences) is small by word2vec standards; the resulting embeddings reflect distributional patterns only within our constructed sentences. Conclusions about embedding quality should not be over-generalised. The Transformer in this work is a single untrained layer; it demonstrates the mechanism of contextualisation but not the full power of deep pre-trained models.

6 Conclusions

We have illustrated — with executable code and five concrete experiments — the fundamental representational difference between Word2Vec and Transformer encoders:

- **Context-independence of Word2Vec.** Word2Vec always maps the same token to the same vector (cross-context cosine = 1.00). This is efficient but creates an irreducible error for polysemous terms ubiquitous in clinical language.
- **Context-sensitivity of Transformers.** Self-attention produces distinct representations per sentence context, with the degree of separation depending on how semantically distinct the two contexts are (greatest for lead/ECG vs. lead/poison).
- **Attention as a disambiguation mechanism.** Visualising attention weights provides direct interpretability: the model attends to semantically relevant context tokens, effectively performing lexical disambiguation.
- **Shared limitation — negation.** Both models fail to represent negation at sentence level, a critical limitation for clinical NLP. This motivates dedicated negation-aware architectures or post-processing.
- **Medical informatics relevance.** Given the prevalence of polysemous terms in medical text, Transformer-based models (BERT, BioBERT, ClinicalBERT) are strongly preferred over Word2Vec for tasks requiring semantic precision such as NER, relation extraction, and clinical document classification.

7 References

- [1] Mikolov, T., Chen, K., Corrado, G., Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.
- [2] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

- [3] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805.
- [4] Lee, J., Yoon, W., Kim, S., et al. (2020). BioBERT: A pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4), 1234–1240.
- [5] Chapman, W. W., Bridewell, W., Hanbury, P., Cooper, G. F., Buchanan, B. G. (2001). A simple algorithm for identifying negated findings and diseases in discharge summaries. *Journal of Biomedical Informatics*, 34(5), 301–310.
- [6] Měhlík, R., Sojka, P. (2010). Software framework for topic modelling with large corpora. In *LREC 2010 Workshop on New Challenges for NLP Frameworks*, pp. 45–50.